

WEEK-4 PRACTICAL

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

CODE:

```
GNU nano 8.7.1 ProcessParent.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int pid[3], status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    for (int i = 0; i < 3; i++)
    {
        pid[i] = fork();

        if (pid[i] == 0)
        {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                   i + 1, getpid(), getppid());
            sleep(i + 1);
            exit(10 + i);
        }
    }

    printf("All 3 child processes have been created.\n");

    printf("Using wait():\n");
    wait(&status);
    printf("wait() detected child PID finished with exit status %d\n",
           WEXITSTATUS(status));

    printf("Using waitpid():\n");

    for (int i = 0; i < 3; i++)
    {
        waitpid(pid[i], &status, 0);

        waitpid(pid[i], &status, 0);
        printf("waitpid() detected child PID %d finished with exit status %d\n",
               pid[i], WEXITSTATUS(status));
    }

    printf("Parent process completed.\n");

    return 0;
}
```

OUTPUT:

```
saicharan_reddi@SAICHARANREDDY:~$ nano ProcessParent.c
saicharan_reddi@SAICHARANREDDY:~$ gcc ProcessParent.c -o ProcessParent
saicharan_reddi@SAICHARANREDDY:~$ ./ProcessParent
Parent Process Started
Parent PID: 551
Child 1 started. PID = 552, Parent PID = 551
Child 2 started. PID = 553, Parent PID = 551
All 3 child processes have been created.
Using wait():
Child 3 started. PID = 554, Parent PID = 551
wait() detected child PID finished with exit status 10
Using waitpid():
waitpid() detected child PID 552 finished with exit status 10
waitpid() detected child PID 553 finished with exit status 11
waitpid() detected child PID 554 finished with exit status 12
Parent process completed.
saicharan_reddi@SAICHARANREDDY:~$ |
```

REPORT: The Multiple child processes were created successfully and their completion was synchronized using wait() and waitpid().

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

CODE:

```
GNU nano 8.7.1 ChildProcess.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int pid = fork();

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        sleep(10);

        printf("Parent terminating...\n");
    }

    return 0;
}
```

OUTPUT:

```
saicharan_reddi@SAICHARANREDDY:~$ nano ChildProcess.c
saicharan_reddi@SAICHARANREDDY:~$ gcc ChildProcess.c -o ChildProcess
saicharan_reddi@SAICHARANREDDY:~$ ./ChildProcess
Parent process: PID = 630
Child PID = 631
Child process: PID = 631
Child process terminating...
Parent terminating...
saicharan_reddi@SAICHARANREDDY:~$ |
```

REPORT: The zombie process was successfully observed, and using wait() in the parent process prevented the child from remaining as a zombie.

PS -EL:

```
saicharan_reddi@SAICHARANREDDY:~$ ps -el
F S  UID      PID     PPID  C PRI  NI ADDR  SZ  WCHAN  TTY          TIME CMD
4 S   0         1         0  0  80   0 -   6034 -      ?           00:00:00 systemd
5 S   0         2         1  0  80   0 -   795 -      hvc0        00:00:00 init-systemd(Ub
0 S   0         6         2  0  80   0 -   795 -      hvc0        00:00:00 init
4 S   0        46         1  0  79  -1 -  12600 -      ?           00:00:00 systemd-journal
4 S  989        78         1  0  80   0 -   5604 -      ?           00:00:00 systemd-resolve
4 S   0        81         1  0  80   0 -   8916 -      ?           00:00:00 systemd-udev
1 S   0       163         1  0  80   0 -  21371 -      ?           00:00:00 snapfuse
1 S   0       164         1  0  80   0 - 113536 -      ?           00:00:01 snapfuse
1 S   0       167         1  0  80   0 -  39837 -      ?           00:00:00 snapfuse
1 S   0       168         1  0  80   0 -  39837 -      ?           00:00:00 snapfuse
4 S   0       196         1  0  80   0 -    722 -      ?           00:00:00 chronyd-starter
4 S   0       197         1  0  80   0 -   1118 -      ?           00:00:00 cron
4 S  995       198         1  0  80   0 -   2204 -      ?           00:00:00 dbus-daemon
4 S   0       202         1  0  80   0 -  11024 -      ?           00:00:00 networkd-dispat
4 S   0       205         1  0  80   0 - 555123 -      ?           00:00:00 snapd
4 S   0       207         1  0  80   0 -   4609 -      ?           00:00:00 systemd-logind
4 S   0       210         1  0  80   0 - 448011 -      ?           00:00:00 wsl-pro-service
4 S  100       226         1  0  80   0 -  55137 -      ?           00:00:00 rsyslogd
4 S   0       229         1  0  80   0 -   1373 -      tty1        00:00:00 agetty
4 S   0       253         1  0  80   0 -  30865 -      ?           00:00:00 unattended-upgr
4 S  988       254       196  0  80   0 -   5106 -      ?           00:00:00 chronyd
1 S  988       273       254  0  80   0 -   3024 -      ?           00:00:00 chronyd
5 S   0       395         2  0  80   0 -    796 -      ?           00:00:00 SessionLeader
5 S   0       396       395  0  80   0 -    800 -      ?           00:00:00 Relay(397)
4 S 1000       397       396  0  80   0 -   1567 do_wai pts/0        00:00:00 bash
4 S   0       398         2  0  80   0 -   2162 -      ?           00:00:00 login
4 S 1000      441         1  0  80   0 -   5589 -      ?           00:00:00 systemd
5 S 1000      443       441  0  80   0 -   5729 -      ?           00:00:00 (sd-pam)
4 S 1000      471       398  0  80   0 -   1534 poll_s pts/1        00:00:00 bash
0 R 1000      641       397  0  80   0 -   1798 -      pts/0        00:00:00 ps
saicharan_reddi@SAICHARANREDDY:~$ |
```

REPORT: The process table was successfully observed using ps -el, and the zombie process was identified by the Z state.

